

Program – 1

Aim – Practice and Study of LEX/YACC of compiler writing.

Theory –

Flex: A Flex application is used to create .l files (lexical analyzer programs) that define lexical rules for text pattern matching. After writing a .l file, it is executed using the Flex tool, which generates a C program that compiles into an executable, transforming input text based on defined patterns.

Flex is a tool for generating lexical analyzers, scanners, or lexers. Written by Vern Paxson in C, circa 1987, Flex is designed to produce lexical analyzers that is faster than the original Lex program. Today it is often used along with Berkeley Yacc or GNU Bison parser generators. Both Flex and Bison are more flexible, and produce faster code, than their ancestors Lex and Yacc.

In this process, Bison creates a parser from the input file provided by the user. In turn, Flex creates the function `yylex()` from a .l file, which is a file containing rules of lexical analysis. The function `yylex()`, created by Flex automatically, retrieves tokens from the input stream and is invoked by the parser to analyze the text of the input. `yylex()` function Note: This is the main function generated from Flex. It performs the rules in the rule section of the .l file to conduct the lexical analysis

Bison: A Bison application complements Flex by handling grammar and parsing for .l files. After Flex generates a lexical analyzer, Bison creates a parser that processes the tokenized input from the .l file. Together, they build a complete compiler or interpreter, enabling complex syntax analysis and execution of programming languages or custom commands.

Dev C++: Dev C++ is an IDE that helps in executing .l files generated by Flex. After Flex processes the .l file, it produces a C file. Dev C++ can then be used to compile this C file into an executable, allowing the lexical analyzer to run and process input based on the defined patterns. Dev C++ also provides debugging tools, enabling you to identify and fix errors in the generated C code. It offers an integrated environment to streamline the compilation process, making it easier to execute and test the .l file's output efficiently.

(.l) File: A .l file is a source file used in lexical analysis, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c: `lex.yy.c` is the C source code file generated by the Flex (or Lex) tool from a .l file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as '`lex.yy.c`'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap(): The `yywrap()` function in a `.l` (Lex) file is a routine that Lex calls when it reaches the end of the input file. By default, it returns 1, signaling the end of input. If `yywrap()` returns 0, Lex continues scanning with the same rules on more input. This function is useful when dealing with multiple input files or when you want to reset the input state. If your program doesn't require special handling at the end of input, you can simply omit it, as the default behavior is often sufficient.

YACC (Yet Another Compiler Compiler): It serves as a powerful grammar parser and generator. In essence, it functions as a tool that takes in grammar specifications and transforms it into an executable module capable of meticulously structuring input tokens into a coherent syntax tree, aligning seamlessly with the prescribed grammar rules. YACC was developed by Stephen C. Johnson in the 1970s. Initially, the YACC was written in the B programming language and was soon rewritten in C. It was originally designed to be used by Lex. In addition to that, YACC was also rewritten in OCaml, Ratfor, ML, ADA, Pascal, Java, Python, Ruby and Go. The input of YACC in compiler design is the rule or grammar, and the output is a C program.

Relationship between LEX and YACC:-

In compiler design, LEX and YACC work together to automate the process of building language translators. LEX (Lexical Analyzer) is used to generate the lexical analyzer (scanner) that reads the input source code and breaks it into tokens based on patterns defined using regular expressions. YACC (Yet Another Compiler Compiler) generates the syntax analyzer (parser) that takes these tokens as input and checks whether they follow the grammar rules of the language, typically defined using context-free grammar. In short, LEX handles the tokenization phase, while YACC handles the parsing phase, and the two integrate seamlessly—LEX sends recognized tokens to YACC, enabling a structured and efficient compiler workflow.

Aspect	LEX	YACC
Purpose	Tokenizer (Lexical Analysis)	Parser (Syntax Analysis)
Input	Regular expressions (patterns)	Context-free grammar (BNF rules)
Output	Tokens (like ID, NUM, PLUS, etc.)	Parse tree / syntax validation
Role	Breaks input into tokens	Checks token sequence against grammar
Connection	Passes tokens to YACC via <code>yylex()</code>	Calls <code>yylex()</code> to get next token
Error Handling	Detects invalid characters	Detects syntax errors
Generated File	<code>lex.yy.c</code>	<code>y.tab.c</code> (and <code>y.tab.h</code>)

Source Code –

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}
```

```
%%  
[0-9]+ { printf("Welcome\n"); exit(0);  
.  
{ printf("Wrong\n"); exit(0); }  
%%
```

```
int yywrap() {  
    return 1;  
}
```

```
int main() {  
    printf("Enter:");  
    yylex();  
    return 0;  
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex first.l  
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c  
  
C:\Users\Kunal\Desktop\KP>a  
Enter:29  
Welcome  
  
C:\Users\Kunal\Desktop\KP>a  
Enter:Yash  
Wrong  
  
C:\Users\Kunal\Desktop\KP>a  
Enter: .  
Wrong  
  
C:\Users\Kunal\Desktop\KP>|
```

Program – 2

Aim – Write a program to count number of tokens in a string

Software Used – Flex, C Compiler (gcc)

Theory – A token refers to a meaningful unit of input that the lexical analyzer identifies during the scanning phase. These tokens can represent keywords, operators, identifiers, literals, or symbols in the source code. The lexer uses regular expressions to define patterns that match specific types of tokens. When a match is found in the input stream, the corresponding token is returned to the parser, often along with an associated value (such as the actual text or a numerical value).

For instance, the string "123" might be recognized as a NUMBER token with a value of 123. In programs that also use tools like Yacc or Bison for parsing, tokens act as a bridge between the lexical analyzer and the parser, helping to structure and interpret the input according to grammar rules. Essentially, tokens are the building blocks of the syntax tree that the parser constructs from the input.

lex.yy.c: lex.yy.c is the C source code file generated by the Flex (or Lex) tool from a `.l` file. When you run Flex on a `.l` file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap(): The 'yywrap()' function in a Lex file signals the end of input processing. It is called when 'yylex()' reaches the end of input. By default, 'yywrap()' returns 1 to indicate that no more input files are available. If you need to process multiple files, 'yywrap()' can be customized to handle additional files or perform cleanup tasks. In most simple applications, returning 1 is sufficient to indicate that no further processing is required

Source code –

```
%{
#include <stdio.h>
#include <stdlib.h>
int tokens = 0;
}%

%%

[a-zA-Z_][a-zA-Z0-9_]* { tokens++; }
[0-9]+ { tokens++; }
[\\t\\n ]+ ;
. ;

%%

int yywrap() {
```

```
    return 1;
}

int main() {
    printf("Enter: ");
    yylex();

    printf("Number of tokens: %d\n", tokens);
    return 0;
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex third.l

C:\Users\Kunal\Desktop\KP>gcc lex.yy.c

C:\Users\Kunal\Desktop\KP>a
Enter : Hi, I'm Yash Pandey, a 3rd-year CSE student at MAIT College. Passionate about tech and
eager to learn more.

Number of tokens: 22

C:\Users\Kunal\Desktop\KP>
```

Program – 3

Aim – Write a program of calculator that can evaluate arithmetic expression with integer

Software Used – Flex, C Compiler (gcc)

Theory – Arithmetic expression evaluation using Lex and Yacc is an important concept in compiler design. Lex serves as a lexical analyzer that breaks input into tokens such as numbers, operators, and parentheses. Yacc acts as a parser, applying grammar rules to evaluate expressions while handling operator precedence and associativity. Together, they process arithmetic inputs like $(8+2)*3$, ensuring accurate results and demonstrating the collaboration of lexical analysis and parsing.

As the expression is scanned, numbers are pushed onto the operand stack, and operators are pushed onto the operator stack according to precedence rules. Parentheses control the order of operations. At the end, the operators are applied in the correct order to compute the final result.

Source code –

```
%{
#include <stdio.h>
#include <stdlib.h>

int num1, num2;
char op;
}%

%%
[0-9]+    {
            if(num1 == 0)
                num1 = atoi(yytext);
            else
                num2 = atoi(yytext);
        }

[+\\-*/]  { op = yytext[0]; }
\\n      { return 0; }
.        ;
%%

int yywrap() { return 1; }

int main() {
    printf("Enter : ");
    yylex();
    switch(op) {
        case '+': printf("Result = %d\\n", num1 + num2); break;
        case '-': printf("Result = %d\\n", num1 - num2); break;
        case '*': printf("Result = %d\\n", num1 * num2); break;
```

```
case '/':
    if(num2 != 0) printf("Result = %d\n", num1 / num2);
    else printf("Error: Division by zero\n");
    break;
default:
    printf("Invalid operator!\n");
}
return 0;
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex third.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c

C:\Users\Kunal\Desktop\KP>a
Enter: 5+3
Result = 8

C:\Users\Kunal\Desktop\KP>a
Enter: 7*4
Result = 28

C:\Users\Kunal\Desktop\KP>a
Enter: 9-2
Result = 7

C:\Users\Kunal\Desktop\KP>a
Enter: 12/3
Result = 4

C:\Users\Kunal\Desktop\KP>|
```

Program – 4

Aim – Write a lex program to find whether a string belongs to a grammar or not

Software Used – Flex, C Compiler (gcc)

Theory – Lex is a widely used tool for generating lexical analyzers, which are programs designed to recognize patterns in input data using regular expressions. In this program, Lex is specifically employed to check whether a given input string conforms to the grammar $S \rightarrow aS \mid bS \mid \epsilon$. This grammar is capable of generating any string made up entirely of the characters a and b, as well as the empty string. The equivalent regular expression for this grammar is $[ab]^*$, which represents zero or more repetitions of a or b. By comparing the input with this expression, the program determines whether the string is valid, meaning it belongs to the defined grammar, or invalid, meaning it does not satisfy the grammar's rules. This makes Lex useful for tasks like syntax validation, language processing, and lexical analysis in compilers

Source code –

```
%{
#include <stdio.h>
#include <string.h>
%}

%%
[a-b]+
{
    printf("String belongs to the grammar\n");
    exit(0);
}

.*
{
    printf("String does NOT belong to the grammar\n");
    exit(0);
}

%%

int main()
{
```



```
    printf("Enter your String: ");  
    yylex();  
    return 0;  
}  
int yywrap()  
{  
    return 1;  
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex four.l  
  
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c  
  
C:\Users\Kunal\Desktop\KP>a  
Enter your String: abba  
String belongs to the grammar  
  
C:\Users\Kunal\Desktop\KP>a  
Enter your String: abc  
String does NOT belong to the grammar  
  
C:\Users\Kunal\Desktop\KP>|
```

Program – 4

Aim – Write a lex program to check if a string include keyword or not

Software Used – Flex, C Compiler (gcc)

Theory – Lex is a powerful tool for generating lexical analyzers that recognize patterns in text using regular expressions. It processes input by matching strings against a set of predefined rules and executes specific actions when a match is found. In this program, Lex is applied to determine whether a given string corresponds to a programming language keyword, such as int, float, if, or else. The rules for these keywords are defined explicitly, and the program checks the input against them. If the input matches one of the defined keywords, it is classified as a keyword; otherwise, it is treated as a non-keyword, which could be an identifier or any other text. This simple demonstration shows the importance of lexical analyzers in compiler design. By recognizing and categorizing tokens such as keywords, identifiers, and symbols, Lex plays a vital role in the initial phase of source code analysis.

Source code –

```
%{
#include <stdio.h>
#include <string.h>
%}

%%

"int"    { printf("Keyword found: int\n"); }
"float"  { printf("Keyword found: float\n"); }
"if"     { printf("Keyword found: if\n"); }
"else"   { printf("Keyword found: else\n"); }

[a-zA-Z_][a-zA-Z0-9_]* { printf("Not a keyword: %s\n", yytext); }

.\n     { /* Ignore other characters */ }

%%

int yywrap() {
    return 1;
}
```

```
int main() {  
    printf("Enter a string: ");  
    yylex();  
    return 0;  
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex five.l  
  
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c  
  
C:\Users\Kunal\Desktop\KP>a  
Enter a string: int x = 10;  
Keyword found: int  
Not a keyword: x  
  
C:\Users\Kunal\Desktop\KP>a  
Enter a string: if else float apple  
Keyword found: if  
Keyword found: else  
Keyword found: float  
Not a keyword: apple  
  
C:\Users\Kunal\Desktop\KP>|
```

Program – 6

Aim – Write a lex program to implement LL1 parser

Software Used – Flex, C Compiler (gcc)

Theory – An LL(1) parser is a predictive top-down parser that reads the input from left to right and constructs a leftmost derivation using only one symbol of lookahead. It relies on a parsing table, derived from the grammar's FIRST and FOLLOW sets, to determine the appropriate production rule.

In the implementation, LEX performs tokenization of the input, while the LL(1) parsing procedure validates whether the token sequence matches the grammar. This illustrates the cooperation between lexical analysis and syntax analysis in the compiler front-end.

Source code –

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char input_str[100];
int i = 0;

void E();
void E_prime();
void T();
%}

%%
id      { strcat(input_str, "id"); }
\+     { strcat(input_str, "+"); }
\n     { return 0; }
[ \t]   ; /* Ignore whitespace */
.       { printf("Invalid character: %s\n", yytext); exit(1); }
%%

int yywrap() {
    return 1;
}

void E() {
    T();
    E_prime();
}

void E_prime() {
```

```

    if (input_str[i] == '+') {
        i++;
        T();
        E_prime();
    }
}

void T() {
    if (input_str[i] == 'i' && input_str[i+1] == 'd') {
        i += 2;
    } else {
        printf("\nError: Expected 'id' at position %d\n", i);
        exit(1);
    }
}

int main() {
    printf("Enter an expression : ");
    input_str[0] = '\0';

    yylex();
    E();

    if (input_str[i] == '\0')
    {
        printf("\nString is accepted!\n");
    } else
    {
        printf("\nString is rejected! Extra characters found at position %d.\n", i);
    }
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex six.1

C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o parser

C:\Users\Kunal\Desktop\KP>parser
Enter an expression : id+id+id
String is accepted!

C:\Users\Kunal\Desktop\KP>parser
Enter an expression : id+x
Invalid character: x

C:\Users\Kunal\Desktop\KP>

```

Program – 7

Aim – Write a lex program to remove left recursion from a grammar

Software Used – Flex, C Compiler (gcc)

Theory – Left recursion in a grammar occurs when a non-terminal appears as the leftmost symbol in its own production, such as $A \rightarrow A\alpha \mid \beta$. This creates problems for top-down parsers like recursive descent and LL(1) parsers, since the parser would repeatedly expand the same non-terminal without consuming any input, leading to infinite recursion. Left recursion can be classified into two types: immediate and indirect. Immediate left recursion occurs when a non-terminal directly refers to itself, while indirect left recursion happens when one non-terminal leads back to itself through another non-terminal. To make grammars suitable for predictive parsing, left recursion must be eliminated. The standard method rewrites a left-recursive production of the form $A \rightarrow A\alpha \mid \beta$ into a right-recursive form: $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$. This transformation ensures that the grammar no longer causes infinite recursion and can be safely used in LL(1) parsing.

In practice, indirect left recursion is removed by systematically replacing productions and then handling any immediate left recursion. Tools like LEX can automate this process by reading grammar rules, detecting left-recursive patterns, and rewriting them into equivalent right-recursive forms. This integration of lexical analysis and grammar transformation makes parsing more efficient and reliable in compiler front-ends.

Source code –

```
%{
#include <stdio.h>
#include <string.h>

char prod[100];
%}

%%
[A-Z]->[A-Z][a-z]* {
    strcpy(prod, yytext);

    char left = prod[0];
    char right[100];
    strcpy(right, prod + 3);

    if (right[0] == left) {
        printf("Left Recursion Detected: %s\n", prod);
```

```

    printf("After removing Left Recursion:\n");
    printf("%c -> %s%c\n", left, right + 1, left);
    printf("%c' -> %c%c' | ε\n", left, right[0], left);
} else {
    printf("No Left Recursion: %s\n", prod);
}
}

```

```

.\n ;
%%

```

```

int main() {
    printf("Enter a production (e.g., A->Aa):\n");
    yylex();
    return 0;
}

```

```

int yywrap() {
    return 1;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex seven.l

C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o leftrec

C:\Users\Kunal\Desktop\KP>leftrec
Enter a production (e.g., A->Aa):
B->Bc
Left Recursion Detected: B->Bc
After removing Left Recursion:
B -> cB'
B' -> BB' | ε

leftrec
Enter a production (e.g., A->Aa):
C->dD
No Left Recursion: C->dD

C:\Users\Kunal\Desktop\KP>|

```

Program – 8

Aim – Write a lex program to do left factoring on a grammar

Software Used – Flex, C Compiler (gcc)

Theory – Left factoring is an important grammar transformation technique used in compiler design to simplify ambiguous productions. The main goal is to restructure the grammar so that it becomes suitable for LL(1) predictive parsers, which require unambiguous choices during parsing.

For example, consider the grammar: $A \rightarrow ab \mid ac$

Here, both productions share the prefix **a**. To remove this ambiguity, we factor out the common prefix and rewrite it as: $A \rightarrow aA', A' \rightarrow b \mid c$

Source code –

```
%{
#include <stdio.h>
#include <string.h>
char left[10], right1[50], right2[50];
}%

%%

[A-Z]    { strcpy(left, yytext); }
"->"    ;
"|"      ;
[a-zA-Z0-9]+ {
    if (strlen(right1) == 0)
        strcpy(right1, yytext);
    else
        strcpy(right2, yytext);
}

\n {
    int i = 0;
    char common[50] = "";

    while (right1[i] && right2[i] && right1[i] == right2[i]) {
        common[i] = right1[i];
        i++;
    }
}
```



```

}
common[i] = '\0';
if (strlen(common) > 0) {
    printf("\nLeft Factoring applied:\n");
    printf("%s -> %s%s\n", left, common, left);
    printf("%s' -> %s | %s\n", left, right1 + i, right2 + i);
} else {
    printf("\nNo left factoring needed:\n");
    printf("%s -> %s | %s\n", left, right1, right2);
}
right1[0] = '\0';
right2[0] = '\0';
}
. ;
%%

int yywrap() {
    return 1;
}

int main() {
    printf("Enter grammar (example: A->ab|ac):\n");
    yylex();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex factor.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o factor
C:\Users\Kunal\Desktop\KP>factor
Enter grammar (example: A->ab|ac):
B->xy|xz

Left Factoring applied:
B -> xB'
B' -> y | z

C:\Users\Kunal\Desktop\KP>factor
Enter grammar (example: A->ab|ac):
C->pq|rs

No left factoring needed:
C -> pq | rs

C:\Users\Kunal\Desktop\KP>

```

Program – 9

Aim – Write a lex and yacc program together for compiler construction include error handling during parsing

Software Used – Flex, C Compiler (gcc)

Theory – The Lex and Yacc programs together form the front end of a simple compiler that can analyze and evaluate arithmetic expressions. Lex (Lexical Analyzer) scans the input stream and divides it into meaningful units called tokens such as numbers, operators, and parentheses. These tokens are then passed to Yacc (Yet Another Compiler Compiler), which performs syntax analysis (parsing) according to the grammar rules defined in the Yacc file. The grammar follows the structure of arithmetic expressions using operator precedence (+, -, *, /) and parentheses. Semantic actions in Yacc compute the result during parsing. The program also includes error handling, which detects invalid syntax and runtime errors like division by zero. This demonstrates the key stages of compiler design — lexical analysis, syntax analysis, and error recovery — in a practical and simplified form.

- Lex for tokenizing numbers and operators
- Yacc for parsing and evaluating
- Error handling for invalid syntax

Source code –

Lex program – calc.l

```
%{  
  
#include "y.tab.h" // include Yacc definitions  
  
#include <stdio.h>  
  
%}  
  
%%  
  
[0-9]+      { yylval = atoi(yytext); return NUMBER; }  
  
[ \t ]+     ;    // ignore whitespace  
  
\n         { return '\n'; }  
  
.          { return yytext[0]; }  
  
%%
```

```

int yywrap() {

    return 1;

}

```

Yacc program – calc.y

```

%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
}%

%token NUMBER

%%

input:
    /* empty */
    | input line
    ;

line:
    '\n'
    | expr '\n'    { printf("Result = %d\n", $1); }
    ;

expr:
    expr '+' term  { $$ = $1 + $3; }
    | expr '-' term { $$ = $1 - $3; }
    | term
    ;

term:
    term '*' factor { $$ = $1 * $3; }
    | term '/' factor {
        if ($3 == 0) {

```

```

        yyerror("Division by zero");
        $$ = 0;
    } else $$ = $1 / $3;
}
| factor
;
factor:
    '(' expr ')' { $$ = $2; }
| NUMBER
;
%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    printf("Enter arithmetic expressions (e.g., 3+5*2):\n");
    yyparse();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex calc.l
C:\Users\Kunal\Desktop\KP>yacc -d calc.y
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c y.tab.c -o calc
C:\Users\Kunal\Desktop\KP>calc
Enter arithmetic expressions (e.g., 3+5*2):
3+5*2
Result = 13

C:\Users\Kunal\Desktop\KP>calc
Enter arithmetic expressions (e.g., 3+5*2):
10/(2-2)
Error: Division by zero

C:\Users\Kunal\Desktop\KP>calc
Enter arithmetic expressions (e.g., 3+5*2):
3+*4
Error: syntax error

```

Program – 10

Aim – Implement Intermediate Code generation for expressions:

- (a) $x = (y + z) \mid k$
- (b) $x = (a + b) * (c - d)$

Software Used – Flex, C Compiler (gcc)

Theory – Intermediate Code Generation is an essential phase in the compiler design process where the high-level source code is translated into an intermediate representation that is easier for the compiler to optimize and later convert into machine code. In this program, Lex performs lexical analysis by identifying tokens such as variables, operators, and parentheses, while Yacc handles the syntax analysis and constructs the intermediate three-address code (TAC). Temporary variables are used to store partial results of complex expressions like $x = (y + z) \mid k$ and $x = (a + b) * (c - d)$. This representation simplifies later stages of compilation and provides a clear step-by-step view of how arithmetic and logical operations are executed internally by a compiler.

Source code –

Lex program – lcg.l

```
%{  
  
#include "y.tab.h"  
  
#include <stdio.h>  
  
%}  
  
%%  
  
[a-zA-Z]    { yylval = yytext[0]; return ID; }  
"="         { return ASSIGN; }  
"("         { return '('; }  
")"         { return ')'; }  
"|"         { return OR; }  
"*"         { return '*'; }  
"+"         { return '+'; }  
"-"         { return '-'; }  
[\\t\\n ]+   ;    // ignore spaces and newlines  
.  
            { return yytext[0]; }  
  
%%  
  
int yywrap() {
```

```
return 1; }
```

Yacc program – lcg.y

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int tempCount = 1;
```

```
char newTemp[5];
```

```
void newTempVar() {
```

```
    sprintf(newTemp, "t%d", tempCount++);
```

```
}
```

```
int yylex();
```

```
void yyerror(const char *s);
```

```
%}
```

```
%token ID ASSIGN OR
```

```
%%
```

```
stmt:
```

```
    ID ASSIGN expr { printf("\nIntermediate Code:\n%s = %s\n", $1, $3); }
```

```
;
```

```
expr:
```

```
    '(' expr ')' OR ID {
```

```
        newTempVar();
```

```
        printf("%s = %s | %c\n", newTemp, $2, $5);
```

```
        $$ = strdup(newTemp);
```

```
    }
```

```
    | '(' ID '+' ID ')' OR ID {
```

```
        newTempVar();
```

```
        printf("%s = %c + %c\n", newTemp, $2, $4);
```

```
        char temp1[5]; strcpy(temp1, newTemp);
```

```
        newTempVar();
```

```
        printf("%s = %s | %c\n", newTemp, temp1, $7);
```

```
        $$ = strdup(newTemp);
```

```

    }
    | '(' ID '+' ID ')' '*' '(' ID '-' ID ')' {
        newTempVar();
        printf("%s = %c + %c\n", newTemp, $2, $4);
        char t1[5]; strcpy(t1, newTemp);
        newTempVar();
        printf("%s = %c - %c\n", newTemp, $7, $9);
        char t2[5]; strcpy(t2, newTemp);
        newTempVar();
        printf("%s = %s * %s\n", newTemp, t1, t2);
        $$ = strdup(newTemp);
    }
;
%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    printf("Enter expression (e.g., x=(y+z)|k or x=(a+b)*(c-d)):\n");
    yyparse();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex icg.l
C:\Users\Kunal\Desktop\KP>yacc -d icg.y
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c y.tab.c -o icg
C:\Users\Kunal\Desktop\KP>icg
Enter expression (e.g., x=(y+z)|k or x=(a+b)*(c-d)):
x=(y+z)|k

Intermediate Code:
t1 = y + z
t2 = t1 | k
x = t2

C:\Users\Kunal\Desktop\KP>icg
Enter expression (e.g., x=(y+z)|k or x=(a+b)*(c-d)):
x=(a+b)*(c-d)

Intermediate Code:
t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3

```

Program – 11

Aim – Consider an expression $((a+b/c) * (d-e))$ and construct a syntax tree

Software Used – Flex, C Compiler (gcc)

Theory – A syntax tree, also called an abstract syntax tree (AST), is a hierarchical representation of the structure of source code based on the language's grammar rules. In this tree, each internal node corresponds to an operator or a construct (such as an expression or statement), while each leaf node represents an operand like a variable, identifier, or constant. The syntax tree captures the logical and grammatical relationships between components of a program, making it easier for the compiler to perform semantic analysis, optimization, and code generation.

Source code –

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <ctype.h>  
typedef struct node {  
    char value[10];  
    struct node *left, *right;  
} Node;  
Node* createNode(char *val) {  
    Node *newNode = (Node*)malloc(sizeof(Node));  
    strcpy(newNode->value, val);  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}  
void printTree(Node *root, int level) {  
    if (root == NULL)  
        return;  
    printTree(root->right, level + 1);  
    for (int i = 0; i < level; i++)  
        printf(" ");  
    printf("%s\n", root->value);  
    printTree(root->left, level + 1);  
}
```



```

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

Node* makeTree(char expr[]) {
    Node *operand[50], *operator[50];
    int topOp = -1, topVal = -1;
    for (int i = 0; expr[i] != '\0'; i++) {
        if (isspace(expr[i])) continue;
        if (isalnum(expr[i])) {
            char val[2] = {expr[i], '\0'};
            operand[++topVal] = createNode(val);
        }
        else if (expr[i] == '(') {
            operator[++topOp] = createNode("(");
        }
        else if (expr[i] == ')') {
            while (topOp >= 0 && strcmp(operator[topOp]->value, "(") != 0) {
                Node *op = operator[topOp--];
                op->right = operand[topVal--];
                op->left = operand[topVal--];
                operand[++topVal] = op;
            }
            topOp--;
        }
        else {
            while (topOp >= 0 && precedence(operator[topOp]->value[0]) >= precedence(expr[i])) {
                Node *op = operator[topOp--];
                op->right = operand[topVal--];
                op->left = operand[topVal--];
                operand[++topVal] = op;
            }
            char val[2] = {expr[i], '\0'};
            operator[++topOp] = createNode(val);
        }
    }
}

```

```

while (topOp >= 0) {
Node *op = operator[topOp--];
op->right = operand[topVal--];
op->left = operand[topVal--];
operand[++topVal] = op;
}
return operand[topVal];
}
}%
%%
.\n ;
%%
int yywrap() { return 1; }
int main() {
char expr[100];
printf("Enter expression (e.g. ((a+b/c)*(d-e))):\n");
scanf("%[^\\n]", expr);
Node *root = makeTree(expr);
printf("\nSyntax Tree (rotated 90 degrees):\n");
printTree(root, 0);
return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex syntree.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o syntree
C:\Users\Kunal\Desktop\KP>syntree
Enter expression (e.g. ((a+b/c)*(d-e))):
((a+b/c)*(d-e))

Syntax Tree (rotated 90 degrees):
      e
    -
  d
*
  c
  /
  b
+
  a

```

Program – 12

Aim – Construct Recursive Decent Parser for the expression $((a+b)/c)*(d-e)$

Software Used – Flex, C Compiler (gcc)

Theory – A Recursive Descent Parser is a type of top-down parser that uses a set of mutually recursive functions to process the input according to a given grammar. Each function corresponds to a non-terminal symbol and is responsible for recognizing the structure defined by that non-terminal. The parser starts from the start symbol and recursively expands the grammar rules, matching input tokens to terminals as it proceeds. If the parser can successfully consume the entire input string without errors, the string is considered syntactically correct. This parsing technique is simple, easy to implement, and forms the basis for many compilers front-end designs.

Source code –

```
%{  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
char expr[100];  
int i = 0;  
  
  
void E();  
void Eprime();  
void T();  
void Tprime();  
void F();  
  
void error() {  
    printf("\nSyntax Error at position %d\n", i + 1);  
    exit(0);  
}  
  
void match(char t) {  
    if (expr[i] == t)  
        i++;  
    else  
        error();  
}  
  
void E() {
```

```

    T();
    Eprime();
}
void Eprime() {
    if (expr[i] == '+') {
        match('+');
        T();
        Eprime();
    } else if (expr[i] == '-') {
        match('-');
        T();
        Eprime();
    } }
void T() {
    F();
    Tprime();
}
void Tprime() {
    if (expr[i] == '*') {
        match('*');
        F();
        Tprime();
    } else if (expr[i] == '/') {
        match('/');
        F();
        Tprime();
    } }
void F() {
    if (expr[i] == '(') {
        match('(');
        E();
        match(')');
    } else if ((expr[i] >= 'a' && expr[i] <= 'z') || (expr[i] >= 'A' && expr[i] <= 'Z')) {
        i++;
    } else {

```

```

        error();
    } }
}%
%%
.\n ;
%%
int yywrap() {
    return 1; }
int main() {
    printf("Enter expression (e.g. ((a+b)/c)*(d-e)):\n");
    scanf("%s", expr);
    E();
    if (expr[i] == '\0')
        printf("\nExpression is syntactically correct.\n");
    else
        printf("\nSyntax Error! Extra characters found.\n");
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex rdp.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o rdp
C:\Users\Kunal\Desktop\KP>rdp
Enter expression (e.g. ((a+b)/c)*(d-e)):
((a+b)/c)*(d-e)

Expression is syntactically correct.

C:\Users\Kunal\Desktop\KP>rdp
Enter expression (e.g. ((a+b)/c)*(d-e)):
(a+b))

Syntax Error at position 6

```

Program – 13

Aim – Write a program to find out the leading of Non Terminals in a grammar

Software Used – Flex, C Compiler (gcc)

Theory – In compiler design, for a given non-terminal A,

The LEADING of a non-terminal symbol represents the set of terminals that can appear at the beginning of any string derived from that non-terminal. It is used in constructing operator precedence and predictive parsing tables. This program reads grammar productions and recursively finds terminals that can appear first in derivations. If a production begins with a non-terminal, the function explores its LEADING symbols.

Formally,

If $A \Rightarrow a\alpha^*$, where a is a terminal and α is any string (possibly empty), then $a \in \text{LEADING}(A)$. It is used in constructing predictive parsing tables and operator precedence grammars.

Source code –

```
%{
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char grammar[10][10];
char leading[10][10];
int n;

void addLeading(char nonterm, char symbol) {
    int idx = nonterm - 'A';
    int len = strlen(leading[idx]);
    for (int i = 0; i < len; i++) {
        if (leading[idx][i] == symbol)
            return;
    }
    leading[idx][len] = symbol;
    leading[idx][len + 1] = '\0';
}

void findLeading(char nonterm) {
    for (int i = 0; i < n; i++) {
        if (grammar[i][0] == nonterm) {
            for (int j = 2; grammar[i][j] != '\0'; j++) {
                if (!isupper(grammar[i][j])) {
                    addLeading(nonterm, grammar[i][j]);
                    break;
                }
            }
            else if (isupper(grammar[i][j])) {
                findLeading(grammar[i][j]);
            }
        }
    }
}
```

```

        int idx = grammar[i][j] - 'A';
        for (int k = 0; leading[idx][k] != '\0'; k++)
            addLeading(nonterm, leading[idx][k]);
        break;
    } } } } }

}%}
%%
.;
%%

int yywrap() { return 1; }

int main() {
    printf("Enter number of productions: ");
    scanf("%d", &n);
    printf("Enter the grammar productions (e.g., E=E+T):\n");

    for (int i = 0; i < n; i++) {
        scanf("%s", grammar[i]);
        leading[i][0] = '\0';
    }
    for (int i = 0; i < n; i++)
        findLeading(grammar[i][0]);

    printf("\nLEADING of Non-Terminals:\n");
    for (int i = 0; i < n; i++)
        printf("LEADING(%c) = { %s }\n", grammar[i][0], leading[grammar[i][0] - 'A']);
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex leading.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a

Enter number of productions: 5
Enter the grammar productions (e.g., E=E+T):
E=TQ
Q=+T
T=FZ
F=(E)
Z=*F

LEADING of Non-Terminals:
LEADING(E) = { ( }
LEADING(Q) = { + }
LEADING(T) = { ( }
LEADING(F) = { ( }
LEADING(Z) = { * }

```

Program – 14

Aim –Write a program to implement shift reduce parsing in a string

Software Used – Flex, C Compiler (gcc)

Theory – A Shift-Reduce Parsing is a type of bottom-up parsing technique used to analyse and recognize the syntactic structure of an input string based on a given grammar. It builds the parse tree from the leaves (input symbols) upward to the start symbol. The parser uses a stack to store grammar symbols and repeatedly performs two main operations: Shift, which pushes the next input symbol onto the stack, and reduce, which replaces a sequence of symbols on the stack (a handle) with the corresponding non-terminal according to a production rule. This process continues until the entire input is parsed and the stack contains only the start symbol, indicating successful parsing.

Source code –

```
%{
#include <stdio.h>
#include <string.h>
char stack[50], expr[50];
int top = -1, i = 0;

void push(char c) {
    stack[++top] = c;
    stack[top + 1] = '\0';
}
void pop() {
    if (top >= 0)
        stack[top--] = '\0';
}
void check() {
    if (top >= 1 && stack[top] == 'd' && stack[top - 1] == 'i') {
        pop();
        stack[top] = 'E';
        printf("REDUCE -> E\n");
        printf("Stack: %s\tInput: %s\n", stack, &expr[i + 1]);
    }
    while (top >= 2) {
        if ((stack[top - 2] == 'E' && stack[top - 1] == '+' && stack[top] == 'E') ||
            (stack[top - 2] == 'E' && stack[top - 1] == '*' && stack[top] == 'E')) {
            stack[top - 2] = 'E';
            top -= 2;
            stack[top + 1] = '\0';
            printf("REDUCE -> E\n");
            printf("Stack: %s\tInput: %s\n", stack, &expr[i + 1]);
        } else { break; } }
}
}
```



```

%%
.;
%%
int yywrap() { return 1; }
int main() {
    printf("Enter input string (use id for identifiers, e.g. id+id*id): ");
    scanf("%s", expr);
    printf("\nSHIFT REDUCE PARSING\n");
    printf("-----\n");
    while (i < strlen(expr)) {
        push(expr[i]);
        printf("SHIFT -> %c\n", expr[i]);

        if (expr[i] == 'd' && i > 0 && expr[i - 1] == 'i') {
            printf("SHIFT -> id\n");
            check();
        } else if (expr[i] == '+' || expr[i] == '*') {
        } else {
            check();
        } i++;
    }
    check();
    printf("\nString Accepted!\n");
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex shiftreduce.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a

Enter input string (use id for identifiers, e.g. id+id*id): id*id+id

SHIFT REDUCE PARSING
-----
SHIFT -> i
SHIFT -> d
SHIFT -> id
REDUCE -> E
Stack: E          Input: *id+id
SHIFT -> *
SHIFT -> i
SHIFT -> d
SHIFT -> id
REDUCE -> E
Stack: E*E        Input: +id
REDUCE -> E
Stack: E          Input: +id
SHIFT -> +
SHIFT -> i
SHIFT -> d
SHIFT -> id
REDUCE -> E
Stack: E+E        Input:
REDUCE -> E
Stack: E          Input:

String Accepted!

```

Program – 15

Aim – Write a program to show all the operations of a stack

Software Used – Flex, C Compiler (gcc)

Theory – A stack is a linear data structure that follows the Last In First Out (LIFO) principle, meaning the element inserted last is removed first. It supports basic operations like push (insert), pop (remove), peek (view top element), and display. Stacks are widely used in function calls, expression evaluation, memory management, and undo mechanisms. Here we implement stack operations using Flex for input tokenizing and C program for stack logic..

Source code –

```
% {  
#include <stdio.h>  
#include <stdlib.h>  
#define SIZE 5  
    int stack[SIZE], top = -1;  
    int yywrap(void) {  
        return 1;  
    }  
    void push(int x) {  
        if (top == SIZE - 1)  
            printf("Stack Overflow\n");  
        else {  
            top++;  
            stack[top] = x;  
            printf("%d pushed\n", x);  
        } }  
    void pop() {  
        if (top == -1)  
            printf("Stack Underflow\n");  
        else {  
            printf("%d popped\n", stack[top]);  
            top--;  
        } }  
    void peek() {  
        if (top == -1)  
            printf("Stack is Empty\n");  
        else  
            printf("Top element: %d\n", stack[top]);  
    }  
    void display() {  
        if (top == -1)  
            printf("Stack is Empty\n");  
        else {
```

```

                printf("Stack elements: ");
                for (int i = top; i >= 0; i--)
                    printf("%d ", stack[i]);
                printf("\n");
            } }
        % }
    %%
push[ ]+[0-9]+ { push(atoi(yytext + 5)); }
pop { pop(); }
peek { peek(); }
display { display(); }
exit { printf("Exiting...\n"); exit(0); }
[ \t\n] ;
. { printf("Invalid Command\n"); }
%%
int main() {
    printf("Stack Operations (Commands: push <num>, pop, peek, display, exit)\n");
    yylex();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex stack.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a
Stack Operations (Commands: push <num>, pop, peek, display, exit)
push 10
10 pushed
push 20
20 pushed
push 30
30 pushed
display
Stack elements: 30 20 10
peek
Top element: 30
pop
30 popped
pop
20 popped
display
Stack elements: 10
pop
10 popped
pop
Stack Underflow
exit
Exiting...

```



```

        for (k = 0; subResult[k] != '\0'; k++) {
            if (subResult[k] != '#')
                addToResultSet(result, subResult[k]);
            else
                foundEpsilon = 1;
        }
        if (!foundEpsilon)
            break;
        j++;
    } } } } }
void addToResultSet(char* result, char val) {
    int k;
    for (k = 0; result[k] != '\0'; k++)
        if (result[k] == val)
            return;
    result[k] = val;
    result[k + 1] = '\0';
}
int main() {
    char ch;
    char result[20];
    printf("Enter number of productions: ");
    scanf("%d", &n);
    printf("Enter the productions (Format: A->aB or A-># for epsilon):\n");
    for (int i = 0; i < n; i++)
        scanf("%s", grammar[i]);
    printf("\nFIRST sets:\n");
    for (int i = 0; i < n; i++) {
        strcpy(result, "");
        ch = grammar[i][0];
        findFirst(result, ch);
        printf("FIRST(%c) = { ", ch);
        for (int j = 0; result[j] != '\0'; j++)
            printf("%c ", result[j]);
        printf("}\n");
    } return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex f.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a
Enter number of productions: 3
Enter the productions (Format: A->aB or A-># for epsilon):
S->AB
A->a
B->b

FIRST sets:
FIRST(S) = { a }
FIRST(A) = { a }
FIRST(B) = { b }

```

Program – 17

Aim – Write a program to check whether a grammar is operator precedence.

Software Used – Flex, C Compiler (gcc)

Theory – An Operator Precedence Grammar (OPG) is a type of context-free grammar where no two adjacent nonterminal appear on the right-hand side of any production, and no production derives ϵ (epsilon). These grammars are used to build operator precedence parsers, which can efficiently parse arithmetic expressions based on precedence and associativity rules of operators. To check if a grammar is operator precedence, we verify:

1. No production has adjacent non-terminals.
2. No production contains ϵ .

Source code –

```
% {
#include <stdio.h>
#include <string.h>
#include <ctype.h>
    char grammar[10][10];
    int n;
    int yywrap(void) {
        return 1;    }
    int isOperatorPrecedenceGrammar() {
        for (int i = 0; i < n; i++) {
            char *prod = grammar[i];
            if (strchr(prod, '#')) {
                printf("Production %s contains epsilon (#) b Not operator precedence.\n", prod);
                return 0;
            }
            for (int j = 3; prod[j + 1] != '\0'; j++) {
                if (isupper(prod[j]) && isupper(prod[j + 1])) {
                    printf("Production %s has adjacent non-terminals %c%c b Not operator
precedence.\n",
                                prod, prod[j], prod[j + 1]);
                    return 0;
                }
            }
        }
        return 1;    } %
}
%%
[A-Z]->[A-Za-z|+*/()-]+ {
    strcpy(grammar[n], yytext);
    n++;
}
```

```

.\n ;
%%
int main() {
    printf("Enter number of productions: ");
    scanf("%d", &n);
    printf("Enter productions (Format: E->E+T, E->id, etc.):\\n");
    for (int i = 0; i < n; i++)
        scanf("%s", grammar[i]);
    if (isOperatorPrecedenceGrammar())
printf("\\n The grammar is Operator Precedence Grammar.\\n");
else printf("\\n□ The grammar is NOT Operator Precedence Grammar.\\n");
return 0; }

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex op.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a
Enter number of productions: 4
Enter productions (Format: E->E+T, E->id, etc.):
E->E+T
E->T
T->T*F
T->F

The grammar is Operator Precedence Grammar.

```

Program – 18

Aim – Write a program to convert NFA to DFA

Software Used – Flex, C Compiler (gcc)

Theory – A Non-deterministic Finite Automaton (NFA) allows multiple transitions for the same input symbol, including ϵ (epsilon) moves. A Deterministic Finite Automaton (DFA), on the other hand, has exactly one unique transition for each symbol from every state. The process of converting NFA to DFA is known as Subset Construction Method. In this method, every DFA state represents a set of NFA states, and transitions are determined by combining all possible NFA moves for a given symbol.

Source code –

```
% {
#include <stdio.h>
#include <string.h>
#include <ctype.h>
    int n, m;
    char states[10], symbols[10], transition[10][10][10];
    char dfaStates[50][50];
    int dfaCount = 0;
    int yywrap(void) {
        return 1;
    }
    int stateExists(char *state) {
        for (int i = 0; i < dfaCount; i++)
            if (strcmp(dfaStates[i], state) == 0)
                return i;
        return -1;
    }
    void unionStates(char *dest, char *src) {
        for (int i = 0; i < strlen(src); i++) {
            if (strchr(dest, src[i]) == NULL)
                strncat(dest, &src[i], 1);
        }
    }
    void getNextState(char *result, char *current, char symbol) {
        result[0] = '\0';
        for (int i = 0; i < strlen(current); i++) {
            int sIndex = strchr(states, current[i]) - states;
            int symIndex = strchr(symbols, symbol) - symbols;
            if (transition[sIndex][symIndex][0] != '-')
                unionStates(result, transition[sIndex][symIndex]);
        }
    }
}
```



```

%%
.\n ;
%%
int main() {
    printf("Enter number of NFA states: ");
    scanf("%d", &n);
    printf("Enter NFA states: ");
    for (int i = 0; i < n; i++)
        scanf(" %c", &states[i]);
    printf("Enter number of input symbols: ");
    scanf("%d", &m);
    printf("Enter symbols: ");
    for (int i = 0; i < m; i++)
        scanf(" %c", &symbols[i]);
    printf("\nEnter transitions (use '-' for no transition)\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            printf("N4(%c,%c): ", states[i], symbols[j]);
            scanf("%s", transition[i][j]);
        }
    }
    char start;
    printf("\nEnter NFA start state: ");
    scanf(" %c", &start);
    char temp[5] = { start, '\0' };
    strcpy(dfaStates[0], temp);
    dfaCount = 1;

    for (int i = 0; i < dfaCount; i++) {
        for (int j = 0; j < m; j++) {
            char result[50] = "";
            getNextState(result, dfaStates[i], symbols[j]);
            if (strlen(result) == 0)
                strcpy(result, "-");
            if (stateExists(result) == -1 && strcmp(result, "-") != 0) {
                strcpy(dfaStates[dfaCount], result);
                dfaCount++;
            }
        }
    }
    printf("\nDFA Transition Table:\n");
    printf("-----\n");
    printf("STATE\t");
    for (int j = 0; j < m; j++)
        printf("%c\t", symbols[j]);
    printf("\n-----\n");
}

```

```

for (int i = 0; i < dfaCount; i++) {
    printf("%s\t", dfaStates[i]);
    for (int j = 0; j < m; j++) {
        char result[50];
        getNextState(result, dfaStates[i], symbols[j]);
        if (strlen(result) == 0)
            strcpy(result, "-");
        printf("%s\t", result);
    }
    printf("\n");
}
return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex dfa.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c
C:\Users\Kunal\Desktop\KP>a
Enter number of NFA states: 3
Enter NFA states: A B C
Enter number of input symbols: 2
Enter symbols: 0 1

Enter transitions (use '-' for no transition)
NFA(A, 0): AB
NFA(A, 1): -
NFA(B, 0): C
NFA(B, 1): C
NFA(C, 0): -
NFA(C, 1): B

Enter NFA start state: A

DFA Transition Table:
-----
STATE    0      1
-----
A         AB      -
AB        ABC      C
ABC       ABC      BC
C         -       B
BC        C       C

```

Program – 19

Aim – Design and implement a symbol table using a hash table, to perform insertion, lookup, and deletion of identifiers

Software Used – Flex, C Compiler (gcc)

Theory – A Symbol Table is a data structure used by compilers to store information about identifiers such as variables, functions, constants, and objects. It helps the compiler keep track of identifiers, their scope, and attributes. Hash Table is often used to implement symbol tables efficiently, allowing constant-time insertion, lookup, and deletion on average. Each identifier is mapped to a hash index computed from its name. Collisions (two identifiers mapping to the same index) are resolved using chaining (linked lists at each index).

Source code –

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define SIZE 10

struct Symbol {
    char name[20];
    struct Symbol *next;
};
struct Symbol *table[SIZE];
int yywrap(void) { return 1; }

int hash(char *name) {
    int sum = 0;
    for (int i = 0; name[i] != '\0'; i++)
        sum += name[i];
    return sum % SIZE;
}

void insert(char *name) {
    int index = hash(name);
    struct Symbol *newSymbol = (struct Symbol *)malloc(sizeof(struct Symbol));
    strcpy(newSymbol->name, name);
    newSymbol->next = NULL;

    if (table[index] == NULL) {
        table[index] = newSymbol;
    } else {
        struct Symbol *temp = table[index];
        while (temp->next != NULL)
            temp = temp->next;
```

```

    temp->next = newSymbol;
}
printf("Inserted identifier '%s' at index %d\n", name, index);
}

void lookup(char *name) {
    int index = hash(name);
    struct Symbol *temp = table[index];
    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            printf("Identifier '%s' found at index %d\n", name, index);
            return;
        } temp = temp->next;
    }
    printf("Identifier '%s' not found\n", name);
}

void delete(char *name) {
    int index = hash(name);
    struct Symbol *temp = table[index], *prev = NULL;

    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            if (prev == NULL)
                table[index] = temp->next;
            else
                prev->next = temp->next;
            free(temp);
            printf("Identifier '%s' deleted from index %d\n", name, index);
            return;
        }
        prev = temp;
        temp = temp->next;
    }
    printf("Identifier '%s' not found for deletion\n", name);
}

void display() {
    printf("\n--- Symbol Table ---\n");
    for (int i = 0; i < SIZE; i++) {
        struct Symbol *temp = table[i];
        if (temp != NULL) {
            printf("[%d]: ", i);
            while (temp != NULL) {
                printf("%s -> ", temp->name);
                temp = temp->next;
            }
            printf("NULL\n");
        }
    }
}

```

```

    }
}
}%}

%%
insert[ ]+[A-Za-z_][A-Za-z0-9_]* { insert(yytext + 7); }
lookup[ ]+[A-Za-z_][A-Za-z0-9_]* { lookup(yytext + 7); }
delete[ ]+[A-Za-z_][A-Za-z0-9_]* { delete(yytext + 7); }
display          { display(); }
exit             { printf("Exiting...\n"); exit(0); }
[ \t\n]         ;
.               { printf("Invalid command\n"); }
%%

int main() {
    printf("Symbol Table Operations:\n");
    printf("Commands:\n");
    printf("insert <id>\nlookup <id>\ndelete <id>\ndisplay\nnext\n\n");
    yylex();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex symtab.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o symtab
C:\Users\Kunal\Desktop\KP>symtab
Symbol Table Operations:
Commands:
insert <id>
lookup <id>
delete <id>
display
exit

insert x
Inserted identifier 'x' at index 8

insert y
Inserted identifier 'y' at index 9

lookup x
Identifier 'x' found at index 8

delete y
Identifier 'y' deleted from index 9

display

--- Symbol Table ---
[8]: x -> NULL

exit
Exiting...

```

Program – 20

Aim – Write a program to perform:

- a. Constant folding
- b. Dead code elimination
- c. Loop invariant code motion
- d. Analyse and report how optimizations reduce memory usage or execution time →
link to sustainable computing

Software Used – Flex, C Compiler (gcc)

a) Constant Folding:

Theory: Constant Folding is an optimization that evaluates constant expressions at compile time instead of runtime. For example, an expression like $a = 5 + 3$ is replaced with $a = 8$ before execution. This reduces the number of runtime operations, improving efficiency and saving power.

Source code –

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
int yywrap(void) { return 1; }
}%
[0-9]+[ ]*[+\\-*/][ ]*[0-9]+ {
    int a, b, result;
    char op;
    sscanf(yytext, "%d %c %d", &a, &op, &b);
    switch(op) {
        case '+': result = a + b; break;
        case '-': result = a - b; break;
        case '*': result = a * b; break;
        case '/': result = b ? a / b : 0; break; // prevent division by zero
        default: result = 0;
    }
    printf("Original: %s\\nOptimized: %d\\n", yytext, result);
}
.\\n          ;    // ignore other characters
}%
int main() {
    printf("Enter an expression (e.g. 5 + 3): ");
    yylex();
    return 0;
}
```

Output –

```
C:\Users\Kunal\Desktop\KP>flex optimize.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o optimize
C:\Users\Kunal\Desktop\KP>optimize
Enter an expression (e.g. 5 + 3): 10 * 5
Original: 10 * 5
Optimized: 50
```

Sustainability Analysis — Constant Folding

Constant folding improves sustainability by performing constant calculations during compilation instead of runtime. This reduces the number of CPU instructions executed when the program runs, saving both processing time and electrical energy. By minimizing repetitive arithmetic at runtime, it helps the processor operate more efficiently, leading to lower power consumption and less heat generation, which supports energy efficient and sustainable computing

b) Dead Code Elimination:

Theory: Dead Code Elimination removes code that does not affect the program output — typically assignments to variables that are never used later. Removing such instructions reduces memory usage and execution time.

Source code –

```
% {
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_VARS 4096
    char var_name[MAX_VARS][64];
    int var_used[MAX_VARS] = {0};
    int var_cnt = 0;
    int brace_level = 0;
    int returned_at_level = -1;
    int in_function = 0;
    int skip_mode = 0;
    int in_args = 0;
    %
}
%option noyywrap
%option yylineno
%x DECL FUNC ARGS SKIP
ID [a-zA-Z_][a-zA-Z0-9_]*
%%
^"#include".*\n |
^"#define".*\n { ECHO; }
```

```

"int"[ \t]+"main"([ \t]*("[^"]*"))? |
"void"[ \t]+"main"([ \t]*("[^"]*"))? |
"int"[ \t]+ {ID} [ \t]*(" |
"void"[ \t]+ {ID} [ \t]*(" {
    ECHO;
    BEGIN(FUNC);
    in_function = 1;
    brace_level = 0;
    returned_at_level = -1;
    skip_mode = 0;
    in_args = 0;
}
"int"[ \t]+ |
"char"[ \t]+ |
"float"[ \t]+ |
"double"[ \t]+ {
    if (brace_level == 0) {
        ECHO;
    } else {
        BEGIN(DECL);
        fprintf(yyout, "%s", yytext);
    } }
<DECL> {ID} {
    if (var_cnt < MAX_VARS) {
        strcpy(var_name[var_cnt], yytext);
        var_used[var_cnt] = 0;
        var_cnt++;
    } }
<DECL>[ \t]*,[ \t]* { fprintf(yyout, ", "); }
<DECL>[ \t]*=[^,;]* {
    char tmp[64] = "";
    sscanf(yytext + 1, "%63[^,;]", tmp);
    for (int i = 0; i < var_cnt; i++)
        if (strcmp(var_name[i], tmp) == 0) var_used[i] = 1;
    fprintf(yyout, "%s", yytext);
}
<DECL>;
{
    int keep = 0;
    for (int i = 0; i < var_cnt; i++) if (var_used[i]) keep = 1;
    if (keep) {
        int first = 1;
        for (int i = 0; i < var_cnt; i++) {
            if (var_used[i]) {
                if (!first) fprintf(yyout, ", ");
                fprintf(yyout, "%s", var_name[i]);
            }
        }
    }
}

```



```

                                first = 0;
                                }      }
                                fprintf(yyout, "\n");
                                }
                                var_cnt = 0;
                                BEGIN(FUNC);
                                }
<FUNC> "{" {
    ECHO;
    brace_level++;
    if (skip_mode && brace_level >= returned_at_level) skip_mode = 0;
}
<FUNC> "}" {
    ECHO;
    brace_level--;
    if (brace_level == 0) {
        in_function = 0;
        in_args = 0;
        BEGIN(INITIAL);
    }      }
<FUNC> "return" [^;]*;
{
    ECHO;
    if (returned_at_level == -1) {
        returned_at_level = brace_level;
        skip_mode = 1;
    }      }
<FUNC> {ID} {
    if (skip_mode) {
        /* do nothing */
    } else {
        if (strcmp(yytext, "printf") == 0 || strcmp(yytext, "scanf") == 0) {
            fprintf(yyout, "%s", yytext); // ECHO once
            in_args = 1;
        } else {
            ECHO;
            for (int i = 0; i < var_cnt; i++) {
                if (strcmp(var_name[i], yytext) == 0) {
                    var_used[i] = 1;
                    break;
                }      }      }      }      }
<FUNC> "(" {
    if (skip_mode) {
        /* skip */
    } else {
        ECHO;

```

```

        if (in_args) {
            BEGIN(ARGS);
        }
    }
<ARGS> {ID} {
    ECHO;
    for (int i = 0; i < var_cnt; i++) {
        if (strcmp(var_name[i], yytext) == 0) {
            var_used[i] = 1;
            break;
        }
    }
<ARGS>)" {
    ECHO;
    in_args = 0;
    BEGIN(FUNC);
}
<ARGS>. { ECHO; }
<FUNC>. {
    if (skip_mode) {
        if (yytext[0] == '}') {
            unput('}');
            skip_mode = 0;
        } else {
            ECHO;
        }
    }
<FUNC>\n {
    if (!skip_mode) fprintf(yyout, "\n");
}
\n { if (!in_function && !skip_mode) ECHO; }
[ \t]+ ;
. { if (!in_function && !skip_mode) ECHO; }
%%
int main() {
    printf("=== FINAL FINAL Dead Code Eliminator ===\n");
    printf("Paste C code (Ctrl+D to end):\n\n");
    FILE *temp = fopen("temp_clean.c", "w");
    if (!temp) {
        perror("fopen");
        return 1;
    }
    yyout = temp;
    yylex();
    fclose(temp);
    printf("\n--- Cleaned Code ---\n");
    system("cat temp_clean.c");
    printf("\n--- Compiling ---\n");
    if (system("gcc temp_clean.c -o a.out") == 0) {

```

```

        printf("Success! Running...\n");
        system("./a.out");
    } else {
        printf("Failed.\n");
    }
    remove("temp_clean.c");
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex deadopt.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o deadopt
C:\Users\Kunal\Desktop\KP>deadopt
=== FINAL FINAL Dead Code Eliminator ===
Paste C code (Ctrl+D to end):

#include <stdio.h>

int main() {
    int a = 5, b = 3, c;
    printf("Sum = %d\n", a + b);
    return 0;
    c = a + b; // unreachable code
}

--- Cleaned Code ---
#include <stdio.h>

int main() {
    int a, b;
    printf("Sum = %d\n", a + b);
    return 0;
}

--- Compiling ---
Success! Running...
Sum = 8

```

Sustainability Analysis — Dead Code Elimination:

Dead code elimination removes unnecessary statements and unused variables that do not affect program output. This decreases the total instruction count, leading to shorter execution times and reduced memory usage. By preventing the CPU from executing irrelevant code, it cuts down on wasted energy and resource utilization, contributing to sustainable software design and greener computation.

c) Loop Invariant Code Motion:

Theory: Loop Invariant Code Motion moves computations that yield the same result in every loop iteration outside the loop. This avoids redundant calculations, thus improving runtime performance and energy efficiency.

Source code –

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

int yywrap(void) { return 1; }
void optimize_loop_invariant() {
    printf("\nOriginal Code:\n");
    printf("for (i = 0; i < n; i++) {\n");
    printf("    x = a + b;\n");
    printf("    y = x + i;\n");
    printf("}\n");
    printf("\nOptimized Code (after Loop Invariant Code Motion):\n");
    printf("x = a + b; // moved outside loop\n");
    printf("for (i = 0; i < n; i++) {\n");
    printf("    y = x + i;\n");
    printf("}\n");
}
}%}
%%
.\n { optimize_loop_invariant(); }
%%

```

```

int main() {
    printf("Loop Invariant Code Motion Optimization\n");
    printf("-----\n");
    yylex();
    return 0;
}

```

Output –

```

C:\Users\Kunal\Desktop\KP>flex licm.l
C:\Users\Kunal\Desktop\KP>gcc lex.yy.c -o licm
C:\Users\Kunal\Desktop\KP>licm
Loop Invariant Code Motion Optimization
-----

Original Code:
for (i = 0; i < n; i++) {
    x = a + b;
    y = x + i;
}

Optimized Code (after Loop Invariant Code Motion):
x = a + b; // moved outside loop
for (i = 0; i < n; i++) {
    y = x + i;
}

```

Sustainability Analysis — Loop Invariant Code Motion:

Loop invariant code motion enhances sustainability by reducing redundant computations inside loops. By moving constant or unchanging expressions outside the loop, the processor executes fewer instructions per iteration, improving efficiency. This leads to lower computational load, reduced energy consumption, and improved performance per watt, promoting eco-friendly and resource-efficient computing.